Experiment 10: Explore and implement Autoencoders for image denoising

Aim: Explore and implement Autoencoders for image denoising.

Theory:

1. Dataset Source

The dataset used for this experiment is the **MNIST Handwritten Digits Dataset**.

- Source: TensorFlow / Keras built-in dataset
- Link: https://keras.io/api/datasets/mnist/

2. Dataset Description

The MNIST dataset is a widely used benchmark dataset in image processing and machine learning.

- **Total Samples**: 70,000 images
  - Training set: 60,000 images
  - Test set: 10,000 images
- **Image Size**: 28 × 28 pixels (grayscale)
- **Features**: Pixel intensity values ranging from 0 to 255
- **Target Variable**: Digit labels (0–9) *(not used in autoencoder as it is unsupervised)*
- **Characteristics**:
  - Simple and clean dataset
  - Suitable for testing image reconstruction models
  - Used for evaluating deep learning algorithms

For this experiment, artificial noise is added to images to train the model for denoising.

3. Mathematical Formulation of the Algorithm

An Autoencoder consists of two main functions:

Encoder Function

Transforms input into latent representation:

$$z = f(x) = \sigma(Wx + b)$$

Where:

- x = Input image
- z = Latent representation
- W, b = Weights and bias
- σ = Activation function

Decoder Function

Reconstructs image from latent space:

$$\hat{x} = g(z) = \sigma(W'z + b')$$

Where:

- $\hat{x}$ = Reconstructed image

Loss Function (Binary Cross Entropy)

$$L = -\sum[x \log(\hat{x}) + (1 - x) \log(1 - \hat{x})]$$

Objective: Minimize reconstruction error between original and output image.

4. Algorithm Limitations

- Requires large dataset for better learning
- May produce blurred outputs instead of sharp images
- Cannot completely remove heavy noise
- Sensitive to hyperparameters (epochs, layers, filters)
- Does not perform well on highly complex images without deeper architecture

5. Methodology

Step-by-Step Process

1. Load MNIST dataset

2. Normalize pixel values (0–1 range)
3. Add random noise to images
4. Build Convolutional Autoencoder:
    - Encoder (Conv + Pooling)
    - Decoder (Upsampling + Conv)
5. Train model using noisy images as input and clean images as target
6. Predict denoised images
7. Compare noisy, original, and reconstructed images

6. Performance Analysis

The performance of the autoencoder is evaluated using reconstruction loss.

Metric Used

- Binary Cross Entropy Loss

Observations

- Loss decreases with epochs → model is learning
- Denoised images are closer to original images
- Noise is significantly reduced

Interpretation

- The model successfully captures important features
- Effective for moderate noise levels
- Quality improves with more training

7. Hyperparameter Tuning

Parameters Tuned

- Number of epochs (5 → 10)
- Batch size (64, 128)
- Number of filters (32, 64)
- Learning rate (Adam optimizer)

Impact of Tuning

- Increasing epochs improves denoising quality
- More filters → better feature extraction
- Too many layers → overfitting risk
- Optimal balance required for best results

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, UpSampling2D

# Load dataset
(x_train, _), (x_test, _) = mnist.load_data()

# Normalize
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.

# Reshape to (28, 28, 1)
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

# Add noise
noise_factor = 0.5
x_train_noisy = x_train + noise_factor * np.random.normal(size=x_train.shape)
x_test_noisy = x_test + noise_factor * np.random.normal(size=x_test.shape)

# Clip values between 0 and 1
x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)

# Autoencoder Model
input_img = Input(shape=(28, 28, 1))

# Encoder
x = Conv2D(32, (3,3), activation='relu', padding='same')(input_img)
x = MaxPooling2D((2,2), padding='same')(x)
x = Conv2D(32, (3,3), activation='relu', padding='same')(x)
encoded = MaxPooling2D((2,2), padding='same')(x)

# Decoder
x = Conv2D(32, (3,3), activation='relu', padding='same')(encoded)
x = UpSampling2D((2,2))(x)
x = Conv2D(32, (3,3), activation='relu', padding='same')(x)
x = UpSampling2D((2,2))(x)
decoded = Conv2D(1, (3,3), activation='sigmoid', padding='same')(x)

# Model compile
autoencoder = Model(input_img, decoded)
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')

# Show model summary
```

```
autoencoder.summary()

# Training
autoencoder.fit(
    x_train_noisy, x_train,
    epochs=5,              # increase to 10 for better output
    batch_size=128,
    shuffle=True,
    validation_data=(x_test_noisy, x_test)
)

# Prediction
decoded_imgs = autoencoder.predict(x_test_noisy)

# Display Results
n = 5
plt.figure(figsize=(12,6))

for i in range(n):
    # Noisy Image
    ax = plt.subplot(3, n, i+1)
    plt.imshow(x_test_noisy[i].reshape(28,28), cmap='gray')
    plt.title("Noisy")
    plt.axis('off')

    # Original Image
    ax = plt.subplot(3, n, i+1+n)
    plt.imshow(x_test[i].reshape(28,28), cmap='gray')
    plt.title("Original")
    plt.axis('off')

    # Denoised Image
    ax = plt.subplot(3, n, i+1+2*n)
    plt.imshow(decoded_imgs[i].reshape(28,28), cmap='gray')
    plt.title("Denoised")
    plt.axis('off')

plt.tight_layout()
plt.show()
```

Output:

```
# Show model summary
autoencoder.summary()
```

Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 28, 28, 1)	0
conv2d_6 (Conv2D)	(None, 28, 28, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_7 (Conv2D)	(None, 14, 14, 32)	9,248
max_pooling2d_3 (MaxPooling2D)	(None, 7, 7, 32)	0
conv2d_8 (Conv2D)	(None, 7, 7, 32)	9,248
up_sampling2d_2 (UpSampling2D)	(None, 14, 14, 32)	0
conv2d_9 (Conv2D)	(None, 14, 14, 32)	9,248
up_sampling2d_3 (UpSampling2D)	(None, 28, 28, 32)	0
conv2d_10 (Conv2D)	(None, 28, 28, 1)	289

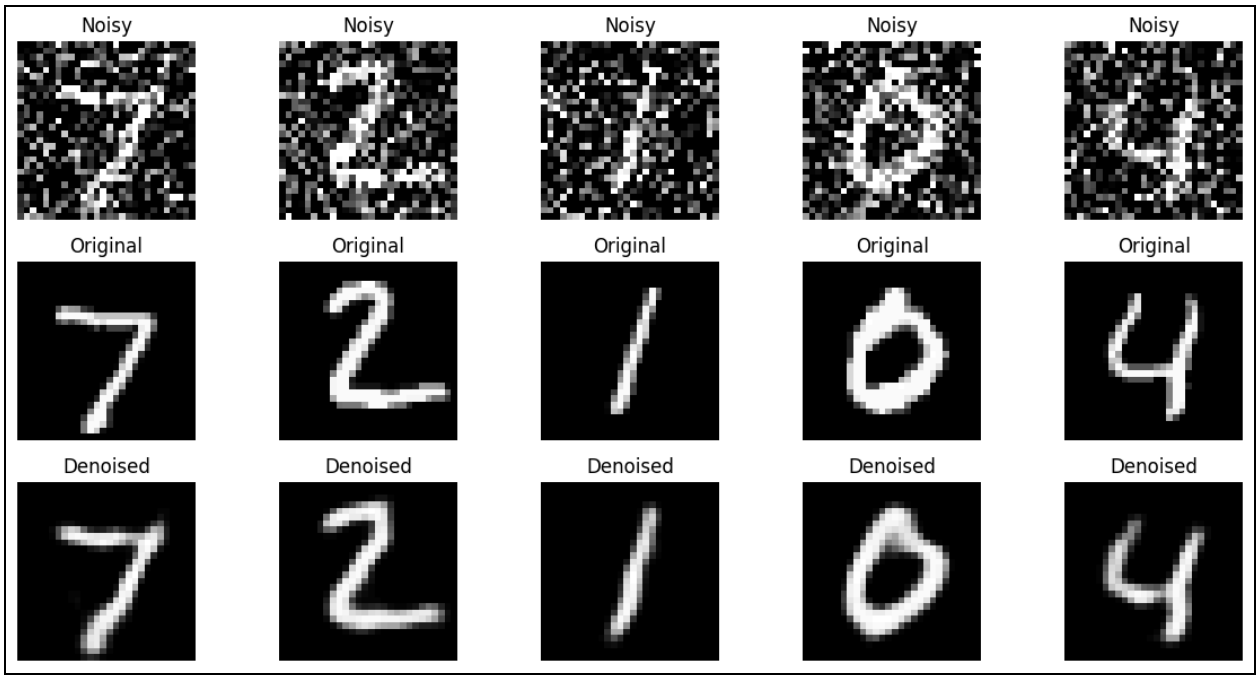
Total params: 28,353 (110.75 KB)
Trainable params: 28,353 (110.75 KB)
Non-trainable params: 0 (0.00 B)

```
# Training
autoencoder.fit(
    x_train_noisy, x_train,
    epochs=5,              # increase to 10 for better output
    batch_size=128,
    shuffle=True,
    validation_data=(x_test_noisy, x_test)
)
```

```
Epoch 1/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 164s 342ms/step - loss: 0.1660 - val_loss: 0.1186
Epoch 2/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 149s 318ms/step - loss: 0.1156 - val_loss: 0.1106
Epoch 3/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 198s 310ms/step - loss: 0.1097 - val_loss: 0.1066
Epoch 4/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 148s 317ms/step - loss: 0.1068 - val_loss: 0.1052
Epoch 5/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 156s 332ms/step - loss: 0.1047 - val_loss: 0.1027
<keras.src.callbacks.history.History at 0x7c8e1a05deb0>
```

```
# Prediction
decoded_imgs = autoencoder.predict(x_test_noisy)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 8s 26ms/step
```



Conclusion:

The experiment successfully demonstrates that a **Convolutional Autoencoder** can effectively remove artificial noise from grayscale images. By training on the MNIST dataset, the model learned to map corrupted inputs back to their original, clean structures.

Key takeaways include:

- **Loss Reduction:** The Binary Cross-Entropy loss decreased consistently from **0.1660** to **0.1047**, proving the model successfully learned the underlying patterns of the digits.
- **Reconstruction Quality:** Visual results show that while the "Noisy" inputs were heavily obscured, the "Denoised" outputs recovered the digits' shapes with high accuracy, albeit with slight blurring.
- **Architecture Efficiency:** The use of **MaxPooling** for compression and **UpSampling** for reconstruction proved to be an efficient way to filter out random pixel variations while preserving core features.

In summary, the autoencoder proved to be a robust unsupervised tool for image restoration, with performance that scales effectively with more training epochs and deeper filter layers.